CSTAM 3.0 · CHALLENGE CSTAM-FELCLOUD

The Resilient IP Optimizer

Solution design, explained step by step: from the problem to the final architecture

What this document is

A walkthrough of the design we reached, one stage at a time. Each stage explains *what* the component does, *how* it works, *why* we need it, and *which alternatives* we rejected and why. Every stage has its own focused diagram; the complete architecture is in the last section.

The final solution in one sentence

One public floating IP points to a virtual IP shared by **two HAProxy + Keepalived gateways**; they route each team subdomain to that **team's own Neutron-managed private IP** on a **Docker host VM (one per hackathon)**, where each team gets an isolated bridge network; a **control plane** (API + IPAM + reconciler + dashboard) provisions, validates, rolls back and tears everything down in a safe order.

Platform: FelCloud OpenStack (region North-Africa) · Version 1 · 14 October 2026

Contents

1. The problem we are solving
2. Stage 1 – The OpenStack foundation
3. Stage 2 – Where a team's sandbox runs
4. Stage 3 – Team networking inside a hackathon host
5. Stage 4 – Redundant HA gateways
6. Stage 5 – Zero-downtime reload and auto-rollback
7. Stage 6 – Provisioning a team
8. Stage 7 – Teardown and IP recycling (IPAM)
9. Stage 8 – Security and isolation
10. Stage 9 – Control plane and admin dashboard
11. **Final solution** – complete architecture, end-to-end flows, requirement coverage
12. Risks, open questions and next steps

How the stages build on each other

Stage	Answers the question	Challenge items it serves
1 Foundation	What do we have on FelCloud and what must exist first?	All (prerequisite)
2 Sandbox runtime	VM, Kubernetes or Docker per team?	Provisioning (15), stress test (15)
3 Team networking	How does a team get its own IP and many apps without port juggling?	Provisioning, IPAM (10), isolation (5)
4 HA gateways	How do we route subdomains with no single point of failure?	HA gateways (15)
5 Reload / rollback	How do we change routes without downtime or broken configs?	Zero-downtime reload & rollback (10)
6 Provisioning	What exactly happens when a team registers?	Provisioning, stress test
7 Teardown / IPAM	How do we release and reuse IPs safely?	IPAM (10), teardown & lifecycle security (10)
8 Security	What stops a team from attacking others or the platform?	Isolation & safeguards (5), security docs
9 Control plane	What drives and observes all of this?	Admin dashboard (10)

1. The problem we are solving

FelCloud hosts high-density hackathons. Today they rely on **static subnets** and **manually edited reverse proxies**. When many teams deploy at once, or a proxy config breaks, the result is downtime and IP conflicts.

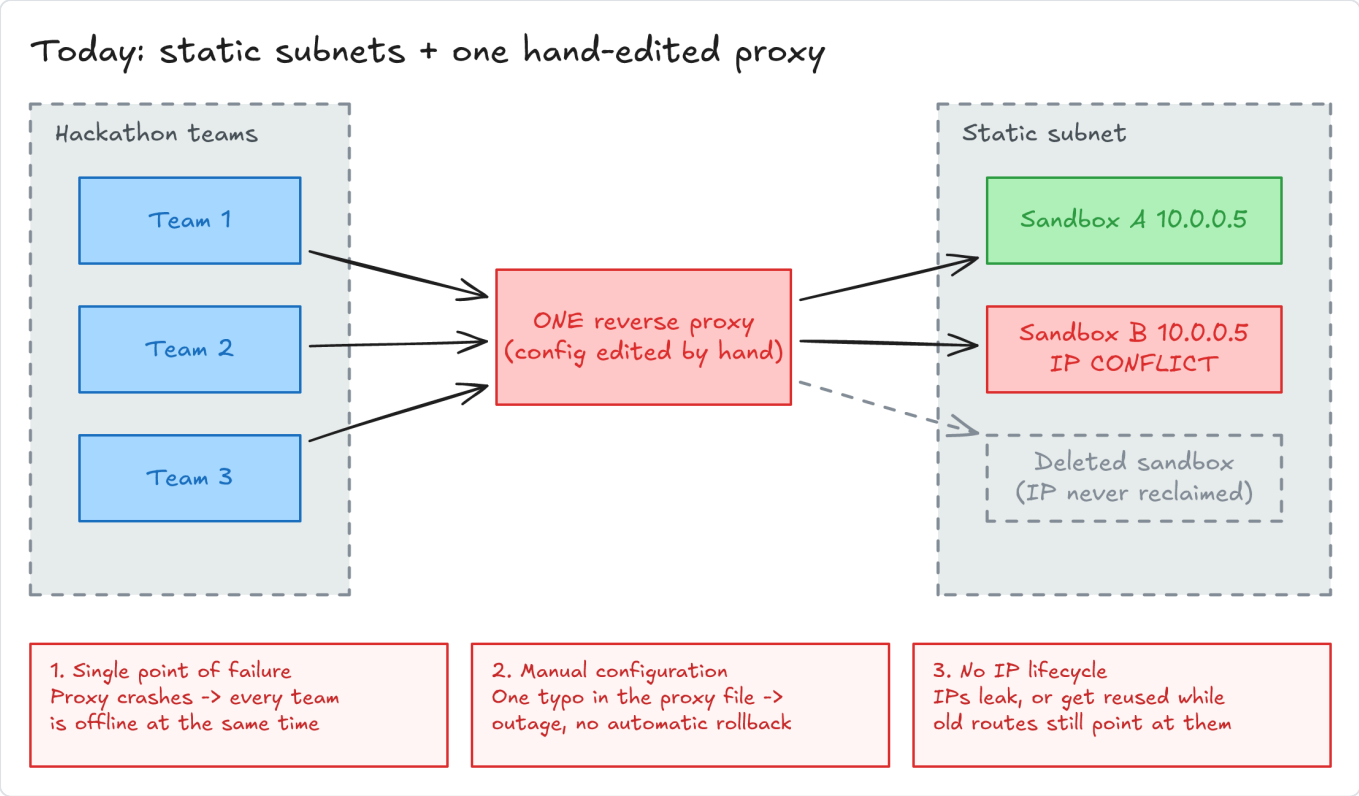


Figure 1 – Today: every team depends on one proxy, edited by hand, in front of IPs nobody manages.

The three root causes (and what the challenge asks for)

Root cause	Symptom	What the challenge requires
Single gateway	Gateway crash = every team offline	Dual redundant HA gateways with instant failover (Phase 1, 15 pts)
Manual configuration	A typo takes the proxy down; no way back	Hot reload + syntax validation + sub-second auto-rollback (Phase 1, 10 pts)
No IP lifecycle	IPs leak, conflict, or get reused while still routed	Automated IPAM with short leases and immediate release (Phase 1, 10 pts); safe teardown (Phase 2, 10 pts)

On top of these, the system must **auto-provision isolated sandboxes via an API** (15 pts), survive **10+ simultaneous registrations** (15 pts), **block inter-sandbox traffic and protect the management layer** (5 pts), and offer an **admin dashboard** (10 pts).

2. Stage 1 – The OpenStack foundation

Purpose: know exactly what FelCloud gives us, and create the few base resources everything else sits on.

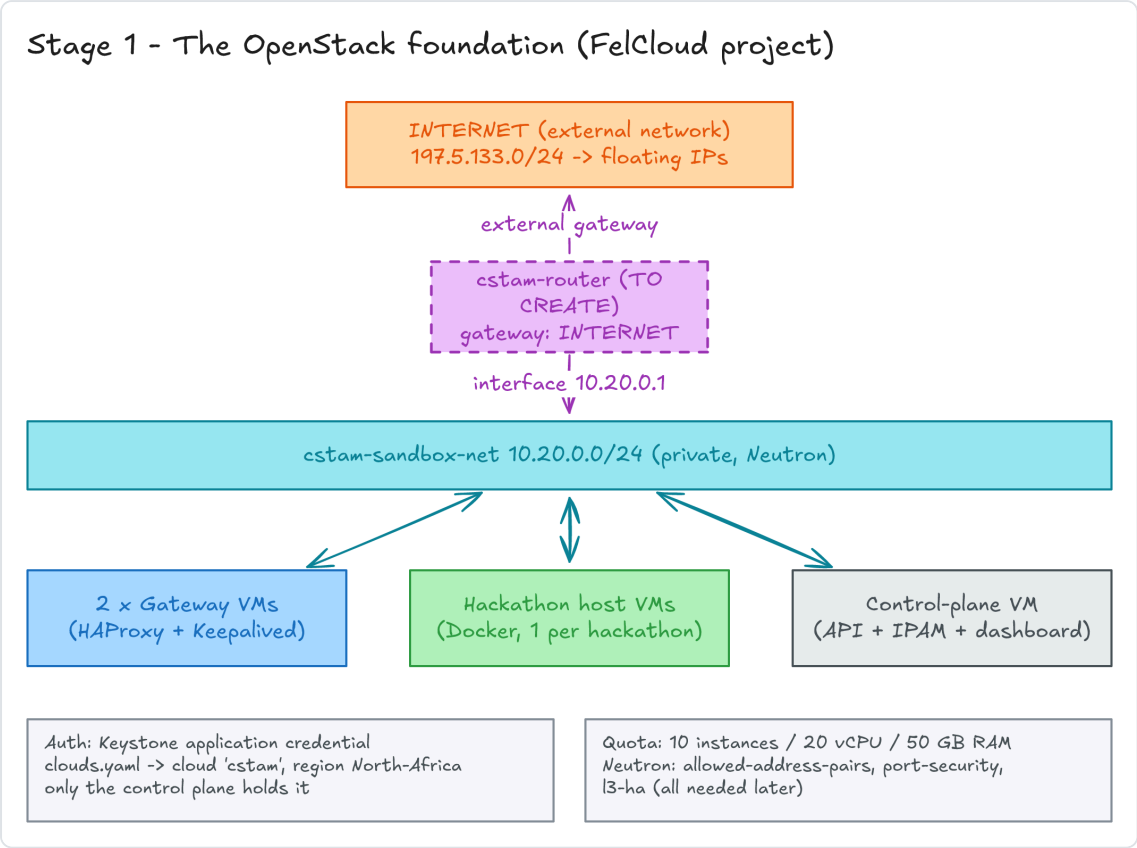


Figure 2 – One private network hosts gateways, hackathon hosts and the control plane; a router connects it to the INTERNET network for floating IPs.

What we verified on FelCloud (with the `openstack` CLI)

- **It is a real OpenStack IaaS**: Keystone v3.14, Nova (compute 2.1), Neutron, Cinder, Glance, Swift, Octavia, Heat, Placement, Skyline dashboard.
- **No container service**: neither Magnum (`container-infra`) nor Zun (`container`) is in the catalog, so containers run on our own VMs.
- **Network**: `cstam-sandbox-net` = `10.20.0.0/24` ; external network `INTERNET` = `197.5.133.0/24` .
- **Neutron extensions we rely on are enabled**: `allowed-address-pairs` , `port-security` , `l3-ha` .
- **Quota**: 10 instances, 20 vCPU, 50 GB RAM, 1000 GB volumes. Two VMs already exist (`sbx-f0699704` , `adam`) and count against it.
- **Access**: a Keystone *application credential* stored in `~/.config/openstack/clouds.yaml` as cloud `cstam` .

Prerequisite found during verification

`cstam-sandbox-net` is **not connected to any router** today (the existing `private_router` has no external gateway and serves another subnet). A floating IP cannot work without one, so step 1 of the build is:

```
openstack router create cstam-router --external-gateway INTERNET
openstack router add subnet cstam-router cstam-sandbox-subnet
```

Why one shared private network

Gateways, hackathon hosts and the control plane all live on `cstam-sandbox-net`. Isolation is not done by splitting networks but by **security groups on every port** (Stage 8). This keeps routing simple: the gateway can reach any team IP directly, and one subnet gives us ~250 addresses to manage in our IPAM.

3. Stage 2 – Where a team's sandbox runs

Purpose: choose the runtime that gives each team an isolated place to deploy, fast, within our quota.

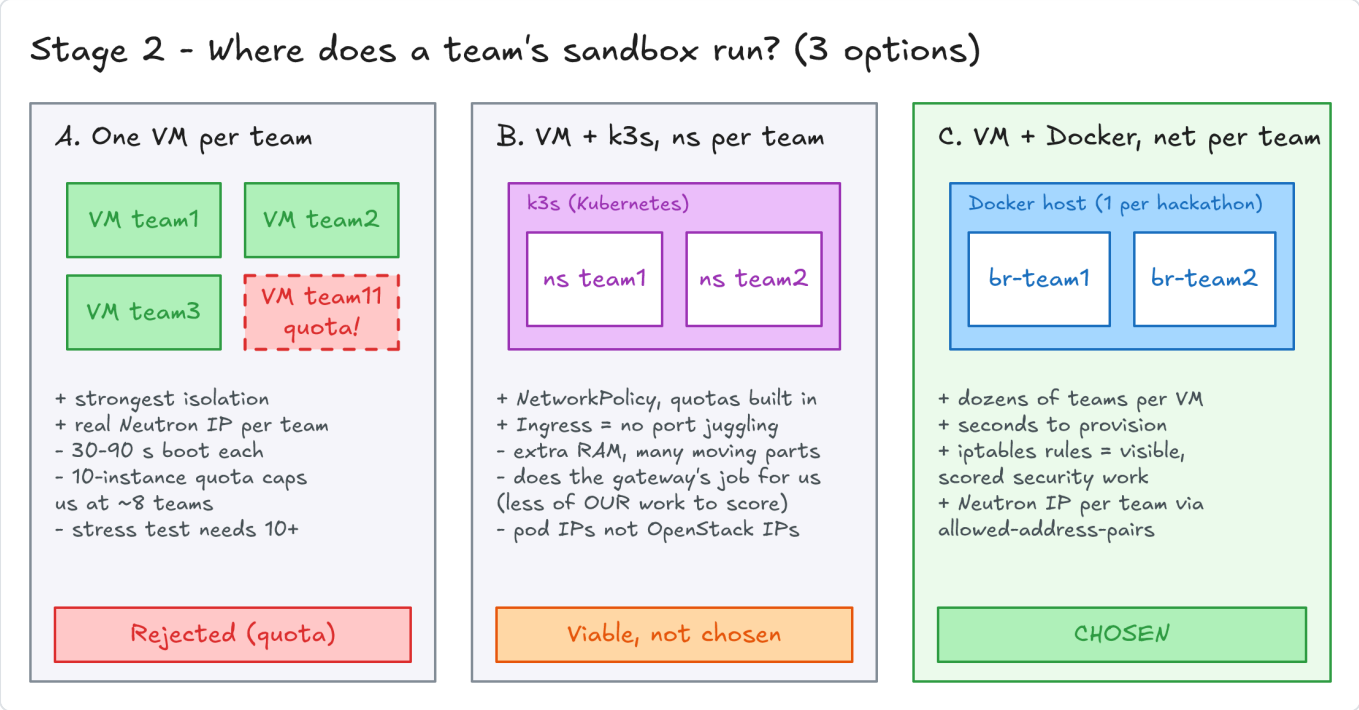


Figure 3 – The three options we evaluated. Option C (one Docker host per hackathon, one network per team) is chosen.

The options in detail

	A. One VM per team	B. VM + k3s, namespace per team	C. VM + Docker, network per team
Isolation	Strongest (own kernel)	Namespaces + NetworkPolicy	Bridge per team + iptables + hardening
Time to provision	30–90 s (boot from volume)	Seconds	Seconds
Fits quota (10 VMs)	No: ~8 teams max, stress test needs 10+	Yes	Yes: dozens of teams per VM
Per-team OpenStack IP	Native	Pod IPs are not Neutron IPs; needs MetalLB tricks	Neutron IP per team via allowed-address-pairs (Stage 3)
Our own scored work visible?	Yes	Less: Ingress/NetworkPolicy do the gateway and isolation work for us	Yes: iptables rules and NAT are explicitly scored items
Complexity / RAM overhead	Low / high per team	Medium / +0.5–1 GB per host	Low / minimal
Verdict	Not chosen (quota); revisit only if FelCloud raises the quota	Viable, not chosen	**Chosen**

Why Docker and not k3s

Docker can meet every security requirement in the challenge: separate bridges block cross-team traffic by default, the `DOCKER-USER` and `INPUT` iptables chains block the metadata service, the host and the management IPs, and container hardening plus per-team cgroup slices handle resource abuse. k3s would add declarative policies, but it also adds RAM, moving parts in a live demo, and it hides exactly the gateway and isolation work the judges score.

Granularity: one host VM per hackathon

Each hackathon gets one Docker host VM (e.g. `G0.basic.4c8g`, roughly 10–15 small team apps). A host VM is a failure domain for its own hackathon only; the platform-wide high availability requirement is about the **gateways** (Stage 4), which we make fully redundant.

4. Stage 3 – Team networking inside a hackathon host

Purpose: give every team a real, OpenStack-managed private IP and let them run as many containers as they want, without routing by port numbers.

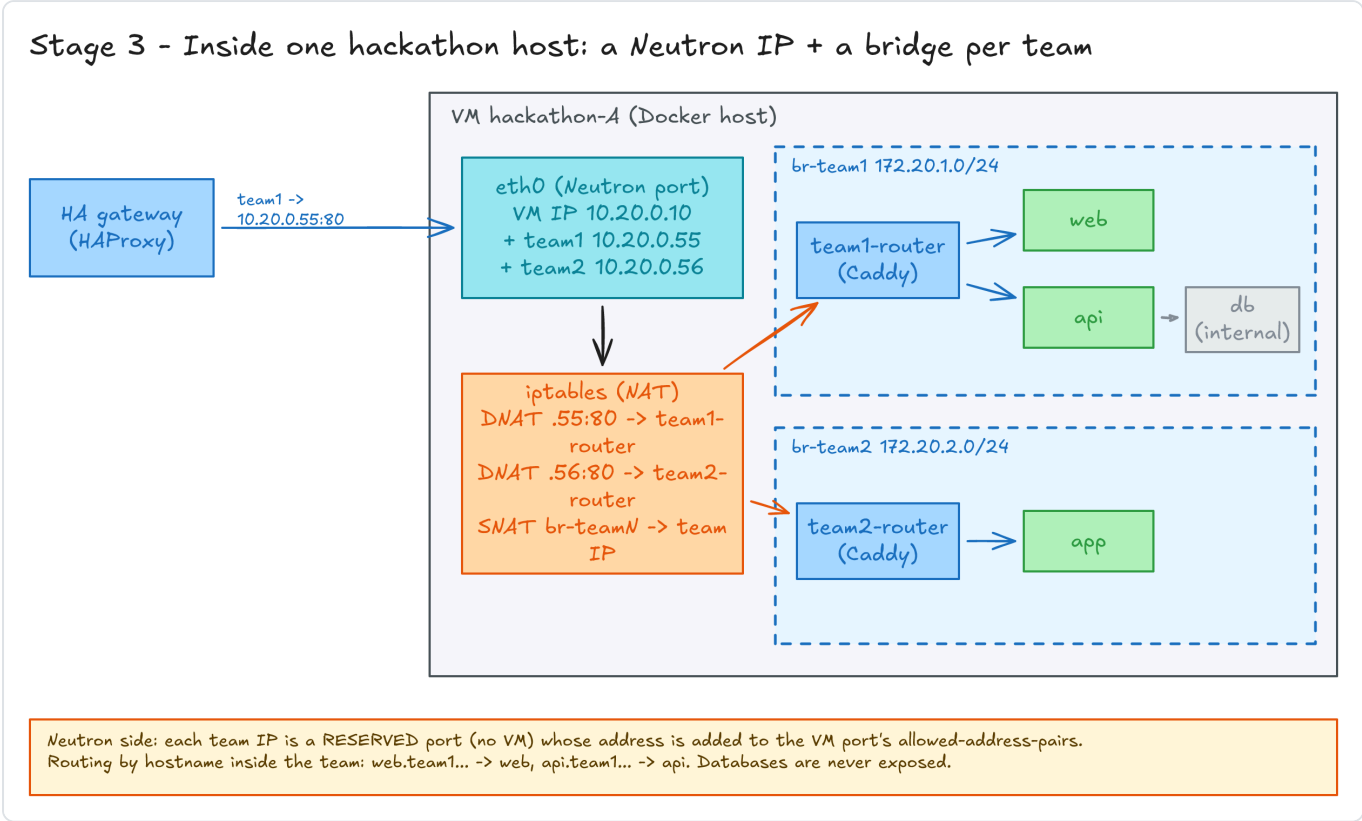


Figure 4 – Each team has its own Neutron IP on the host's network card, its own bridge, and a small router container that routes by hostname.

How it works, step by step

1. **Reserve the team IP in Neutron.** The control plane creates a Neutron port with no VM attached and a fixed IP (e.g. `10.20.0.55`). This is how the IP becomes "managed by OpenStack".
2. **Allow it on the host.** The IP is added to the host VM's port `allowed-address-pairs`. Without this, Neutron's anti-spoofing drops traffic for an IP that does not belong to the port.
3. **Bind it inside the VM.** `ip addr add 10.20.0.55/24 dev eth0`.
4. **Create the team bridge.** A Docker network `br-team1` (e.g. `172.20.1.0/24`), plus a **router container** (Caddy) attached only to that bridge.
5. **NAT.** Inbound: `DNAT 10.20.0.55:80 → team1-router`. Outbound: `SNAT br-team1 → 10.20.0.55`, so all traffic leaving the team carries the team's own IP (useful for logs, rate limiting and audit).
6. **Hostname routing inside the team.** `web.team1…` → `web:3000`, `api.team1…` → `api:8000`. Databases and caches stay internal on the bridge.

Why this design (alternatives we rejected)

Approach	Gateway routes to	Port bookkeeping	OpenStack IP per team
Docker published ports	`VM:8081`, `VM:8082` …	Messy; ports get recycled too	No

Per-VM Traefik + Docker DNS	`VM:80`, then container name	None	No
Literally bridging the Neutron IP into the team network (macvlan)	Each container IP	None	Yes, but needs an allowed pair per container; fragile
Neutron IP per team + NAT + team router (chosen)	`team-IP:80`	None	Yes

> **Why not expose arbitrary TCP ports (5432, 6379...)**
> Hostname routing only works for HTTP (or TLS with SNI). Raw TCP has no hostname, so exposing it through one public IP would bring back per-team public ports, and exposing databases is a security problem. The challenge only asks for subdomain routing.

5. Stage 4 – Redundant HA gateways

Purpose: route every subdomain to the right team IP, through one public IP, with no single point of failure.

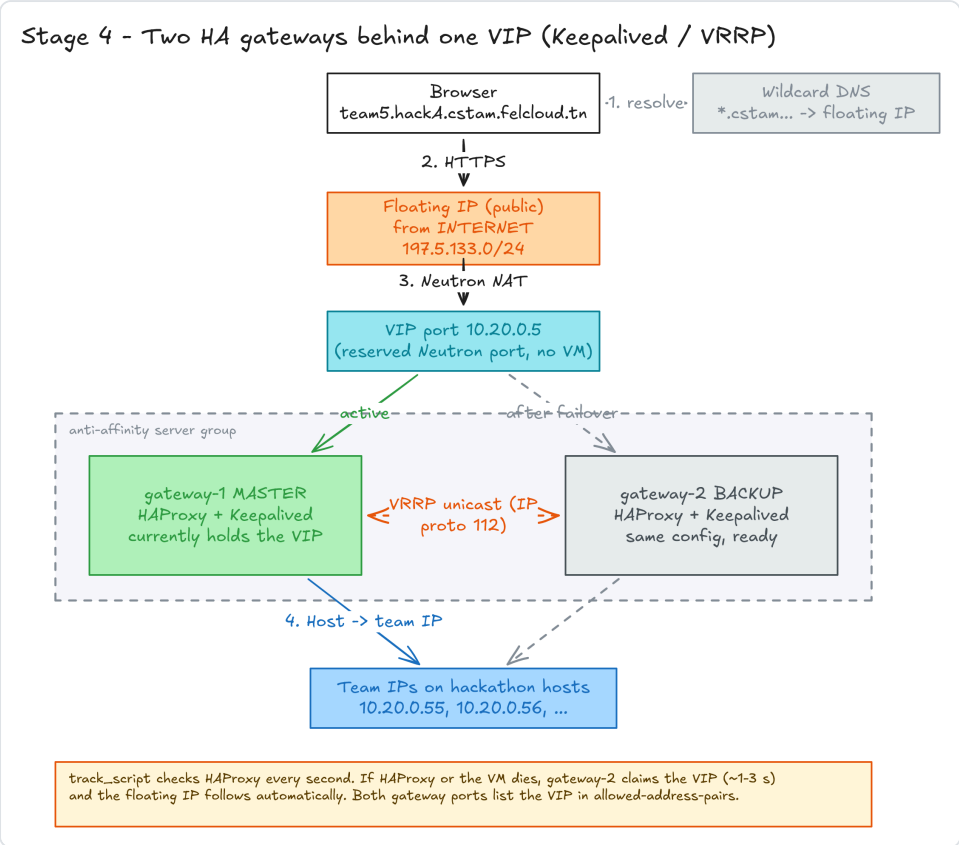


Figure 5 – Two gateways share one VIP. Keepalived moves the VIP to the backup within seconds if the master or its HAProxy dies; the floating IP follows.

How a request flows

1. The browser resolves `team5.hackA.cstam.felcloud.tn`; a single **wildcard DNS** record points all of `*.cstam…` to the floating IP.
2. Neutron NATs the floating IP to the **VIP** `10.20.0.5`, a reserved port owned by no VM.
3. Whichever gateway currently holds the VIP (the VRRP master) receives the request.
4. HAProxy looks up the `Host` header in its **map file** and forwards to the team IP, e.g. `10.20.0.55:80`.

Choice of proxy

	HAProxy	Nginx (OSS)	Traefik	Envoy
Config validation	`haproxy -c`	`nginx -t`	No real test step; bad config is silently ignored	Via control plane
Change routes without reload	**Runtime API + maps**	Only in paid Plus	File/HTTP provider	xDS
Demonstrable rollback	Explicit	Explicit	Hard to show	Complex
Per-team rate limiting (bonus)	Stick-tables, very strong	`limit_req`	Middleware	Yes

Verdict	**Chosen**	Good fallback	Not chosen	Overkill

Choice of failover mechanism

	Keepalived (VRRP)	**Octavia (FelCloud LBaaS)**	**DNS round-robin**
Failover time	~1–3 s (sub-second possible)	Seconds to tens of seconds	Minutes (caching)
Our own work, demo-able crash	Yes: kill gateway-1 live	Black box	No
TLS termination	In HAProxy	Needs Barbican, which FelCloud does not offer	—
Verdict	**Chosen**	Not chosen	Rejected

OpenStack setup that makes VRRP work

1. Reserve the VIP: `openstack port create --network cstam-sandbox-net vip-port`.
2. Add the VIP address to **both** gateway ports' `allowed-address-pairs`.
3. Associate the floating IP with the VIP port.
4. Security group: allow VRRP (IP protocol 112) only between the two gateways; use **unicast** VRRP peers (multicast is often blocked in clouds).
5. Place the gateways in an **anti-affinity server group** so one hypervisor failure cannot take both down.
6. Keepalived `track_script` checks HAProxy every second: if HAProxy dies, the VIP moves even though the VM is alive.

6. Stage 5 – Zero-downtime reload and auto-rollback

Purpose: change routing while traffic flows, and make it impossible for a bad config to stay live.

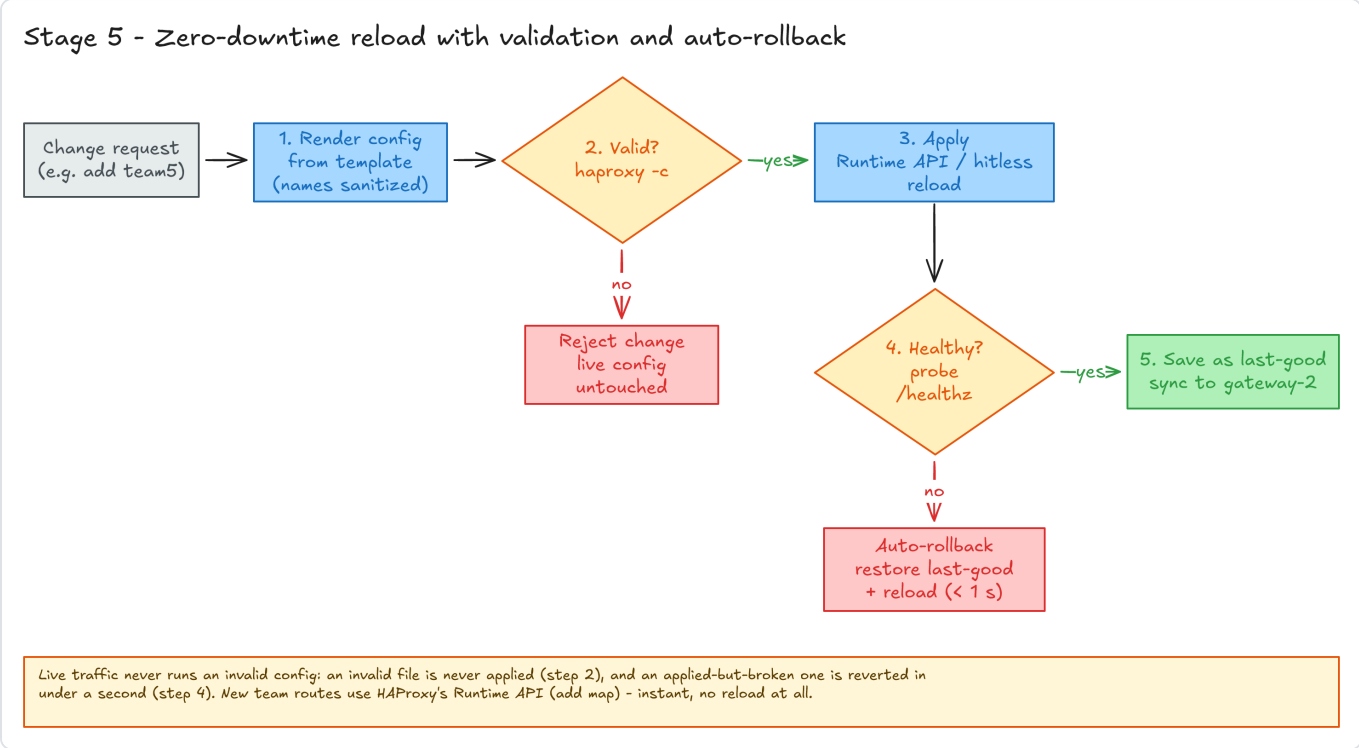


Figure 6 – Two safety gates: validation before applying, health check after applying.

The pipeline

1. **Render** the config from a template. Team names are sanitized to `[a-z0-9-]`, so a name can never inject config.
2. **Validate** with `haproxy -c -f new.cfg`. Invalid → rejected; the live config is never touched.
3. **Apply**: route additions and removals use the **Runtime API** (`add map` / `del map`), which is instant and needs no reload. Structural changes use HAProxy's **hitless reload** (master-worker mode keeps listening sockets).
4. **Health check**: probe `/healthz` and a known route. Failure → restore the last-good config and reload (< 1 s).
5. **Commit**: save as last-good (versioned in the DB) and push the same change to the backup gateway, so failover never lands on stale config.

> **What we will show in the demo**
> Push a deliberately broken config → it is rejected at step 2; push a config that validates but breaks routing → it is rolled back at step 4; meanwhile a load generator shows zero failed requests.

7. Stage 6 – Provisioning a team

Purpose: one API call turns a team registration into a reachable, isolated sandbox, in seconds.

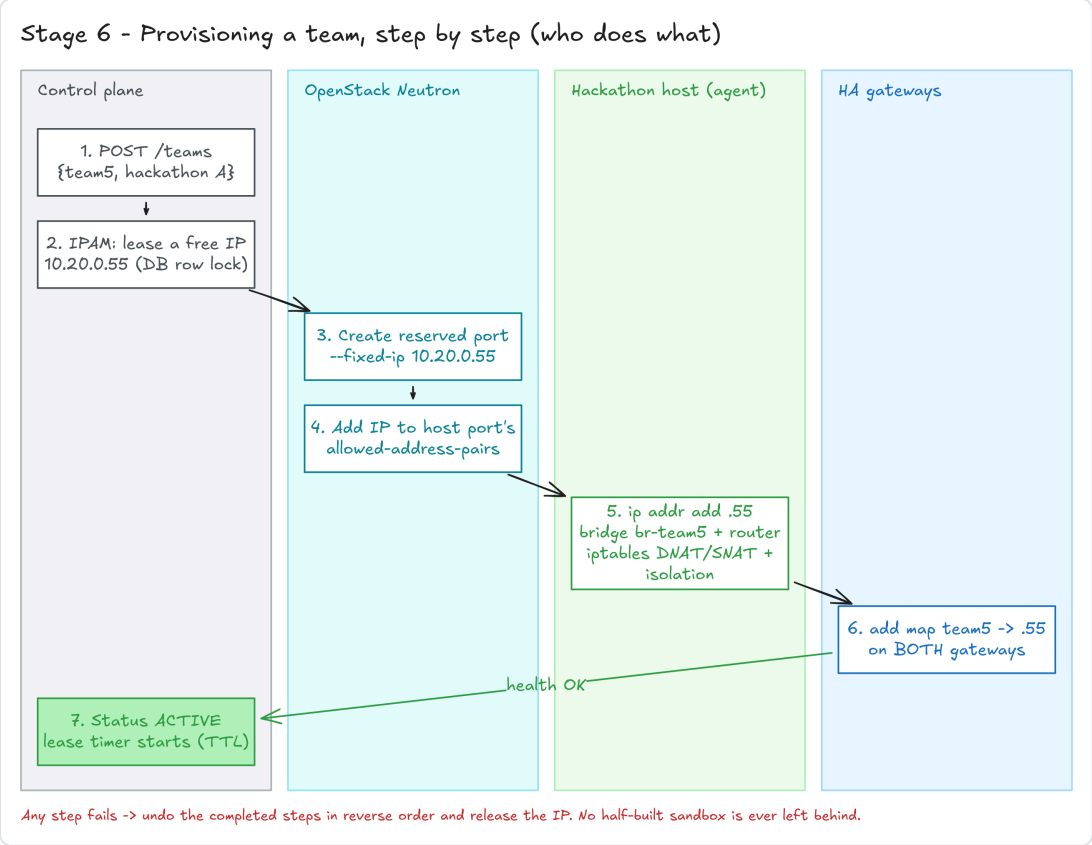


Figure 7 – Who does what when a team registers. Each lane is a separate component.

1. `POST /teams` arrives at the control-plane API.
2. **IPAM** leases a free IP from its pool under a database row lock, so 10+ concurrent registrations never get the same IP.
3. **Neutron**: create the reserved port with that fixed IP.
4. **Neutron**: add the IP to the hackathon host's allowed-address-pairs.
5. **Host agent**: bind the IP, create `br-teamN`, start the team router, install NAT and isolation rules.
6. **Gateways**: add the map entry on both gateways via the Stage 5 pipeline.
7. Health check passes → status **ACTIVE**, lease timer (TTL) starts.

> **Failure handling**
> If any step fails, the completed steps are undone in reverse order and the IP is released, so no half-built sandbox is left behind. The reconciler (Stage 9) also repairs anything left inconsistent by a crash mid-way.

> **Why this is fast enough for the stress test**
> No VM boots during team provisioning: the hackathon host is already running. Each step is one API call or a few kernel operations, so a team is typically live within a few seconds, and steps for different teams run in parallel.

8. Stage 7 – Teardown and IP recycling (IPAM)

Purpose: free resources immediately when a sandbox expires, without ever sending a team's traffic to another team.

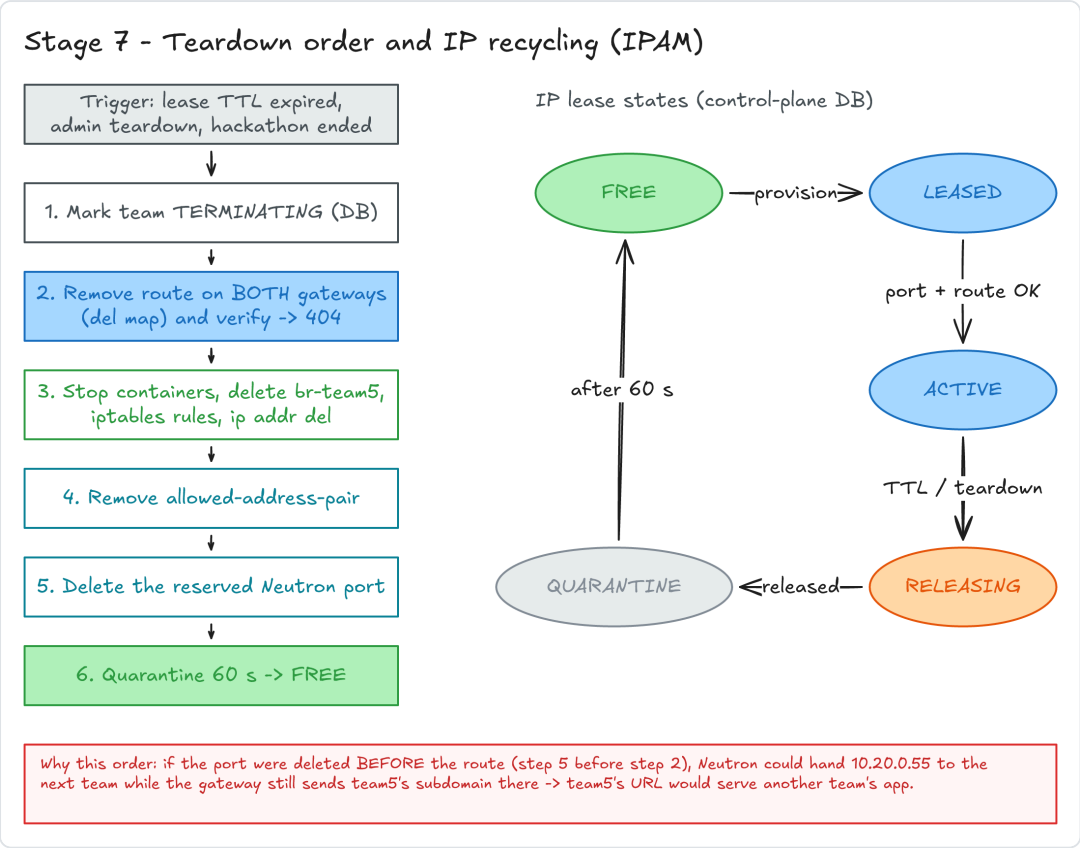


Figure 8 – Left: the strict teardown order. Right: the lifecycle of an IP in our IPAM.

Nothing cleans itself up automatically

Resource	What happens on its own	Consequence
Gateway route	Nothing: HAProxy knows nothing about OpenStack	The route keeps pointing at the old IP
Neutron IP	Freed **immediately** when its port is deleted; can be handed to the next port at once	Next team may get the same IP while the old route still exists
Pre-created port	Deleting a VM only **detaches** a port we created ourselves	The IP leaks unless we delete the port explicitly
DNS	Wildcard record: nothing to delete	Gateway must answer 404 for unknown subdomains

> **The bug the strict order prevents**
> If the port were deleted before the route, Neutron could give `10.20.0.55` to the next team while the gateway still sends `team5…` there. Team 5's URL would then serve another team's app. The rule: **an IP may only return to the pool once nothing points at it**, and a short **quarantine** (60 s) absorbs in-flight connections and caches.

Our IPAM engine on top of Neutron

- Tracks every IP's state: `FREE → LEASED → ACTIVE → RELEASING → QUARANTINE → FREE`.
- Requests **specific** free IPs from Neutron (`--fixed-ip ip-address=…`) so quarantined IPs are never reused early.
- Short leases with TTL; the reconciler expires them and triggers teardown automatically.
- Metrics for the benchmark report: lease time, release time, time from teardown to IP reusable.

9. Stage 8 – Security and isolation

Purpose: make sure a team can only reach its own sandbox, and nobody can reach the platform's management layer.

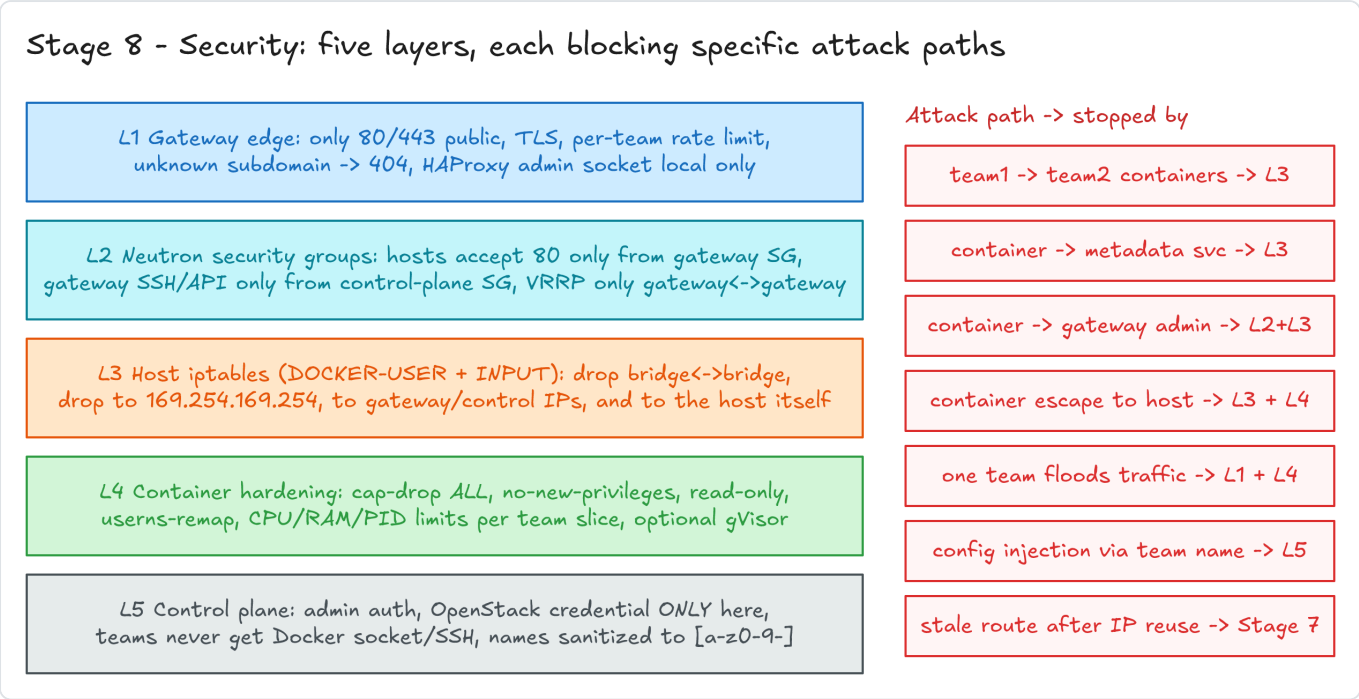


Figure 9 – Defence in depth: five independent layers, each mapped to the attack paths it stops.

Mapping to the challenge

Challenge requirement	Covered by
Block inter-sandbox traffic	L3: separate bridges (Docker isolation chains) + `DOCKER-USER` drop rules; L2: hosts accept traffic only from the gateway security group
Protect the gateway management layer from sandboxes	L2: gateway SSH/Runtime API only from the control-plane security group; L3: drop sandbox traffic to gateway, control-plane and metadata IPs; L1: HAProxy admin socket bound locally
Lifecycle security (sanitize IP allocations)	Stage 7 order + quarantine; reconciler removes orphans
Isolated sandboxes	L4: `cap-drop ALL`, `no-new-privileges`, read-only root, user-namespace remapping, CPU/RAM/PID limits per team slice, optional gVisor
Config integrity	L5: names sanitized; Stage 5 validation and rollback

One finding to fix

The existing `sbx-f0699704` security group is described as "HTTP from gateways" but has **no ingress rules at all**. Our security groups will be created with explicit rules referencing the gateway group (`remote_group_id`).

10. Stage 9 – Control plane and admin dashboard

Purpose: one place that decides, executes, remembers and shows everything.

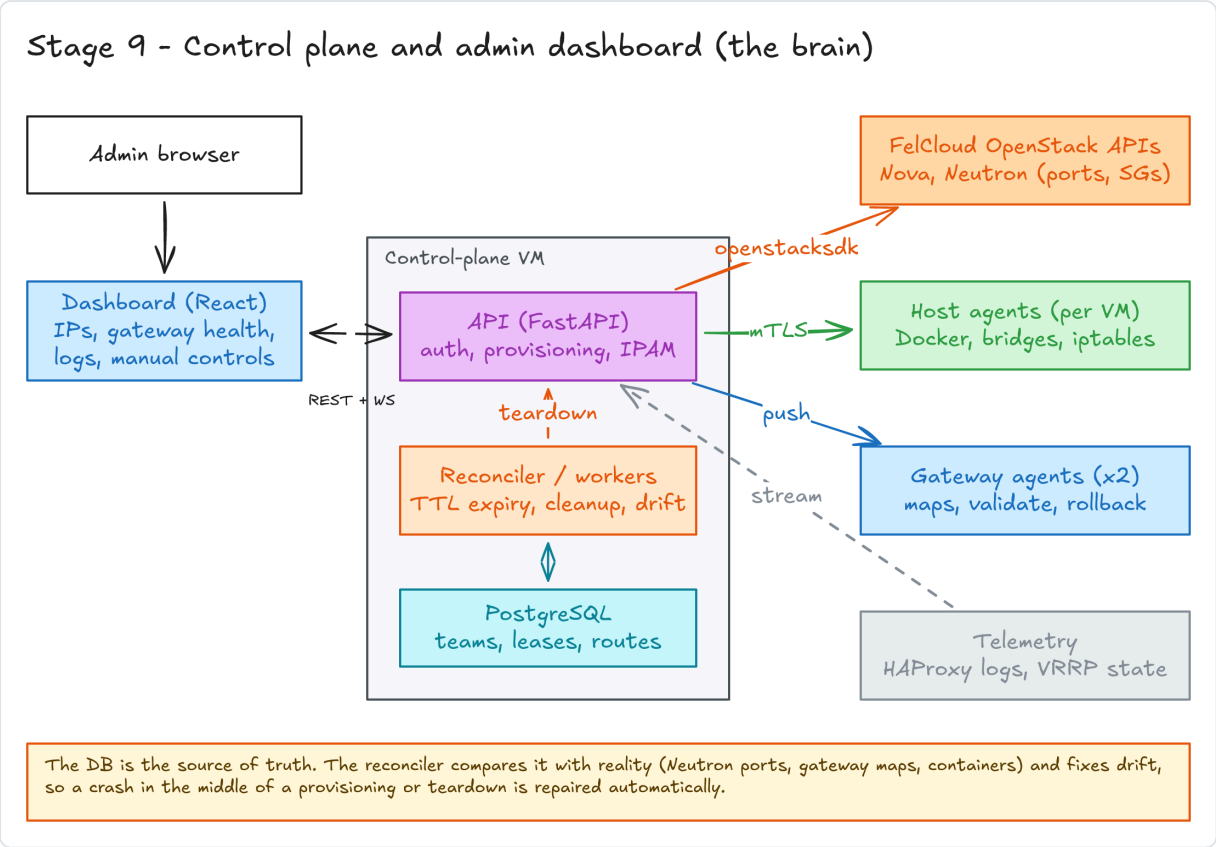


Figure 10 – The API drives OpenStack, the host agents and the gateway agents; the reconciler keeps reality equal to the database.

Component	Role
API (FastAPI)	Auth, team and hackathon CRUD, IPAM, orchestration of Stages 5–7
PostgreSQL	Source of truth: teams, leases, IP states, routes, config versions (for rollback)
Reconciler / workers	Lease expiry, teardown pipeline, drift repair (orphan ports, stale routes, dead containers)
Host agent (per hackathon VM)	Docker networks, router containers, iptables, IP binding; reached over mTLS
Gateway agent (x2)	Map updates, validation, reload, rollback; reports health and VRRP state
Dashboard (React)	Active sandboxes and their OpenStack IPs, gateway health (master/backup), subdomain map, live traffic logs, and buttons for spin-up, teardown, manual failover and manual rollback

Why a reconciler

Provisioning touches four systems. If the control plane crashes halfway, something is left inconsistent. Instead of hoping every failure path is handled, the reconciler continuously compares the database with Neutron, the gateways and Docker, and fixes the difference. This is also what makes the teardown "automated" for the Phase 2 score.

11. Final solution – the architecture we will build

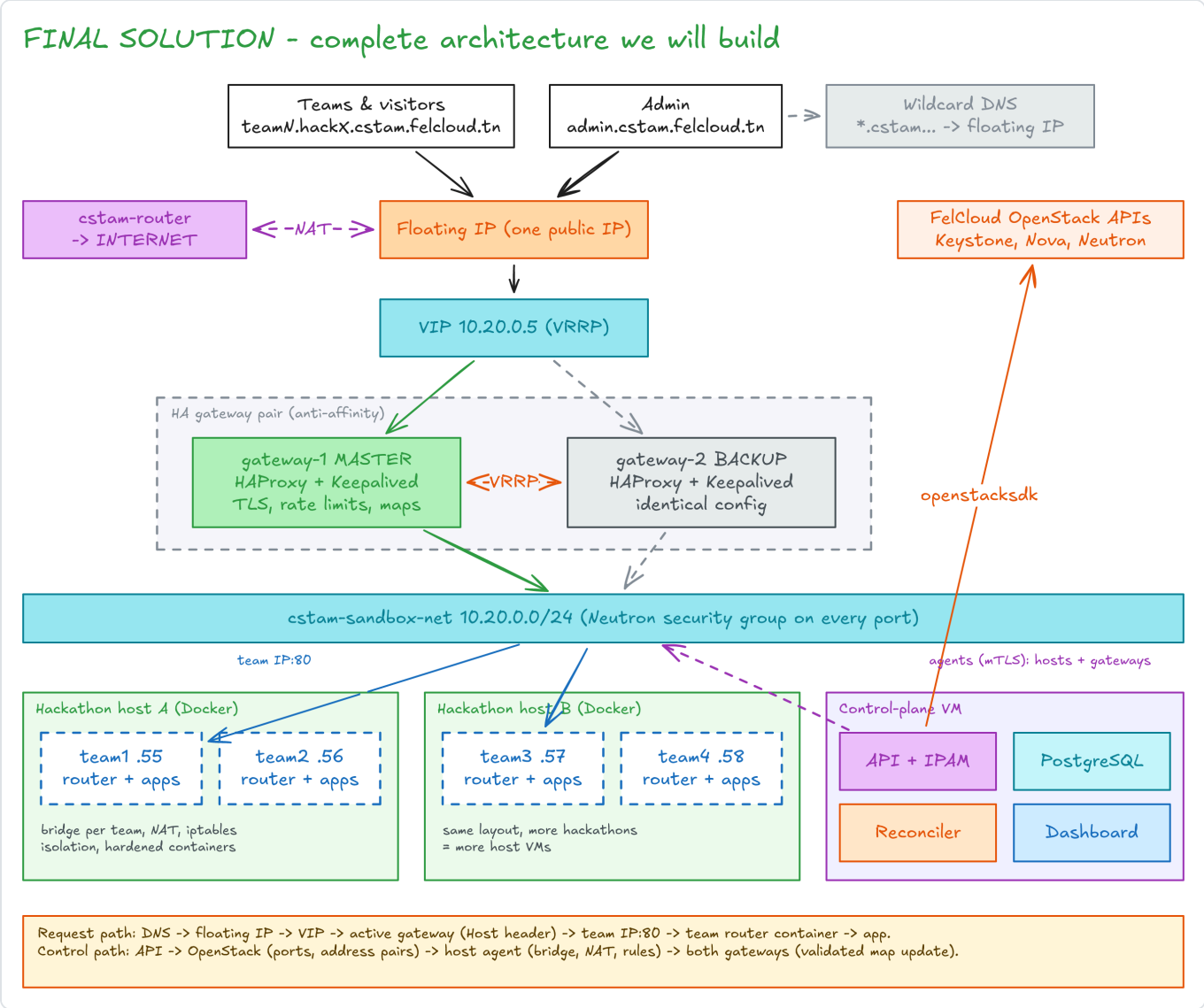


Figure 11 – **Final solution.** All stages combined: one public IP, a redundant gateway pair, Docker hosts per hackathon with a Neutron IP per team, and the control plane.

End-to-end walkthroughs

A visitor opens a team app. DNS (wildcard) → floating IP → VIP on gateway-1 → HAProxy maps `team1.hackA…` to `10.20.0.55:80` → host A DNATs to the team1 router → router picks the app by hostname.

A team registers. API → IPAM lease → Neutron reserved port + allowed pair → host agent builds the bridge, router, NAT and rules → both gateways get a validated map entry → ACTIVE in seconds.

Gateway-1 crashes. Keepalived on gateway-2 stops receiving VRRP adverts → claims the VIP within ~1–3 s → the floating IP follows the VIP → gateway-2 already has identical maps → traffic continues.

A sandbox expires. Reconciler → route removed on both gateways → containers, bridge, rules removed → allowed pair removed → port deleted → IP quarantined, then FREE.

Requirement coverage

Challenge item	Points	Where in the design
Automated OpenStack sandbox provisioning	15	Stages 2, 3, 6
Dual redundant HA gateways	15	Stage 4
Automated IPAM & reclamation	10	Stages 3, 7
Zero-downtime reload & auto-rollback	10	Stage 5
Concurrent provisioning stress test (10+)	15	Stage 6 (DB-locked IPAM, no VM boot per team)
Automated teardown & lifecycle security	10	Stage 7, reconciler
Network isolation & access safeguards	5	Stage 8
Admin dashboard	10	Stage 9
Bonus: TLS / rate limiting / maintenance page	+2 each	HAProxy (wildcard certificate, stick-tables, errorfile/backup server)
Bonus: OpenStack-native orchestration	+2	Optional: Heat templates for the gateway pair and host VMs

Capacity within the current quota

VM	Flavor	Count	vCPU	RAM
Gateways	G0.basic.1c2g	2	2	4 GB
Control plane	G0.basic.2c4g	1	2	4 GB
Hackathon hosts	G0.basic.4c8g	2	8	16 GB
Already existing (sbx, adam)	1c1g, shared 4c4g	2	5	5 GB
Total vs quota		**7 / 10**	**17 / 20**	**29 / 50 GB**

A third hackathon host fits if the stopped `adam` VM is deleted (it still counts against the quota while SHUTOFF).

Why this is the right final design

- **No single point of failure on the shared path**: the only component every team depends on (the gateway) is redundant and fails over automatically.
- **Every team IP is a real Neutron IP** with a lease, a state and a safe release, which is exactly what the IPAM items score.
- **No port juggling**: everything routes by hostname to `IP:80`.
- **Fits the quota** while handling 10+ concurrent teams in seconds.
- **The scored work is ours and visible**: VRRP failover, validated reloads, rollback, iptables isolation, IPAM.

12. Risks, open questions and next steps

Things to test on FelCloud first

Risk	Why it matters	Test
Floating IP on an unbound VIP port	FelCloud runs OVN (we saw a `network:distributed` port); floating IPs on ports owned by no VM must work for the VIP design	Create router, VIP port, floating IP; move the VIP between two test VMs
VRRP between gateways	Needs IP protocol 112 allowed and unicast peers	Two small VMs with Keepalived; kill one, measure failover time
Allowed-address-pairs on host ports	The whole per-team IP design depends on it	Add a secondary IP to a VM, reach it from another VM

Questions for the FelCloud mentors

- Can we get a wildcard DNS record `*.cstam.felcloud.tn`, and API access for a Let's Encrypt DNS-01 challenge (wildcard TLS bonus)?
- Is a quota increase possible (would also allow one-VM-per-team as a premium isolation mode)?
- Who created `sbx-f0699704`, and is there an existing CSTAM tool we should build on?

Build order

1. Router + floating IP + VIP experiment (Stage 1 and the risks above).
2. Gateway pair with Keepalived and HAProxy maps; failover demo (Stage 4).
3. Validation and rollback pipeline (Stage 5).
4. One hackathon host with the per-team network recipe, done by hand, then scripted as the host agent (Stage 3).
5. Control plane: API, IPAM, provisioning and teardown pipelines, reconciler (Stages 6, 7, 9).
6. Security rules and hardening (Stage 8), then the stress test and benchmark report.
7. Dashboard, documentation, demo video.